Complaint Management System – Simple Product Document

Prepared by Satya Mythri Reejinthala

1. What Is It?

The Complaint Management System is a web portal where users can raise complaints and track them, and admins can review and manage those complaints. Each person sees only what their role allows, and the system uses secure login.

Built with: MongoDB, Express.js, React.js, Node.js (MERN), JWT, Nodemailer

2. Problem and Solution

Problem	Solution
Complaints sent by email or paper get lost	One portal where every complaint is recorded
Users cannot see what is happening with their complaint	Users can track their complaints
Admins have no single place to manage complaints	Admin side to review and manage them
Accounts can be misused if security is weak	JWT login, role-based access and a secure OTP password reset

3. Who Uses It? (Personas)

Meena, the User (28): Wants to raise an issue and know someone is working on it.

Suresh, the Admin (45): Responsible for solving complaints and needs one place to see them all.

4. User Stories

ID	User story	Done when...
1	As a user, I want to register and log in so that my account is protected.	A JWT token is given on login and checked on protected pages
2	As a user, I want to submit a complaint so that my issue is recorded.	The complaint is saved against my account
3	As a user, I want to track my complaints so that I know the progress.	I can see only my own complaints
4	As an admin, I want to manage all complaints so that I can resolve them.	Admin-only pages reject normal users
5	As a user, I want to reset my password with an OTP so that I can recover my account.	OTP arrives by email, expires after a set time and works safely
6	As an admin, I want repeated wrong OTP attempts blocked so that attackers cannot guess codes.	Attempts stop after the allowed limit

5. How It Works (Flow)

User registers → Logs in and receives a JWT token → Submits a complaint → Admin sees it on the admin side → Admin manages it → User tracks the progress

Password reset: User asks for reset → OTP is emailed → User enters OTP → System checks expiry and attempt limit → User sets a new password

6. Features by Priority

Priority	Feature	Status
Must have	Login with JWT and separate User and Admin access	Built
Must have	Complaint submission and tracking	Built

Priority	Feature	Status
Should have	OTP password reset with expiry and attempt limit	Built
Could have	Email alerts when a complaint status changes	Future idea
Could have	Complaint categories, priority and a statistics dashboard	Future idea

7. How I Would Measure Success

- Average time taken to resolve a complaint
- Percentage of complaints resolved on time
- How many users successfully reset their password
- How many suspicious OTP attempts were blocked

8. 30-Second Explanation (Say This in an Interview)

"The Complaint Management System is a MERN-stack portal where users raise complaints and track them, and admins manage them. I separated User and Admin permissions on every API endpoint and used JWT for authentication. I also built a secure password reset with OTP emails that expire and block repeated wrong attempts. Next, I would add email alerts and a statistics dashboard."

9. Likely Questions and Short Answers

Why did you build this? Complaints are often lost or untracked, so I wanted a simple portal with clear ownership and visible progress.

What is JWT? A signed token the server gives after login. The user sends it with each request so the server knows who they are without asking for the password again.

What is role-based access? Users and admins have different permissions. For example, only an admin can manage all complaints, and a normal user can see only their own.

Why limit OTP attempts? To stop brute-force guessing. Without a limit, an attacker could try many codes until one works.

What would you improve next? Email alerts on status changes and a dashboard that shows resolution time and complaint categories.